Geometric Algebra (GA)

Daniel Hug

2025

1 Documentation

Overview (high-level initial intros on youtube):

- *https://www.thestrangeloop.com*
- *https://www.youtube.com/@sudgylacmoe*
- *https://www.youtube.com/@bivector*

Sources:

- https://projectivegeometricalgebra.org
- https://terathon.com/blog/poor-foundations-ga.html
- https://bivector.net
- https://www.youtube.com/@bivector
- https://www.youtube.com/@sudgylacmoe
- https://www.youtube.com/watch?v=ItGlUbFBFfc
- https://www.youtube.com/watch?v=v-WG02ILMXA&t=1476s

1.1 Introduction

The ga library is intended to make Geometric Algebra (GA) more accessible by providing functionality for numerical calculations required for simulation of physical systems.

It implements functionality for two- and three-dimensional spaces with Euclidean geometry. Currently, it provides data types and operations to handle the algebras of regular Euclidean geometric algebra ($ega2d, ega3d$) and projective Euclidean geometric algebra ($pga2dp, pga3dp$). The library handles fully populated multivectors, even and odd grade multivectors as well as their components like scalars, vectors, bivectors, trivectors, and pseudoscalars of the corresponding linear spaces efficiently. It also provides operations to work with these objects. It was designed with easy extensibility in mind and can be extended to provide functionality for handling additional algebras, e.g. spacetime algebra (STA), with limited effort.

The main library is a header only library in the ga sub-folder of the library which can be used by including either

```
#include ga/ga_ega.hpp // and/or
#include ga/ga_pga.hpp
```

and by making the corresponding namespaces accessible by stating

```
using namespace hd::ga;      // and
using namespace hd::ga::ega; // either/or
using namespace hd::ga::pga;
```

in your application code. For a simple usage example please have a look either at `ga_test/src/ga_ega_test.cpp` or at `ga_test/src/ga_pga_test.cpp`.

All product expressions for inner, outer, interior and geometric products as well as their regressive counterparts are generated by programs in the subfolder `ga_prdxpr`. User input to configure product generation is provided in the header files of the corresponding algebra as well. The generated coefficient expressions are used in the source code to implement dot products, wedge products, contractions, expansions, geometric products or their regressive counterparts, as well as sandwich products describing rotors and motors of the corresponding algebras. If certain products are required for your applicaton, but not yet implemented in the ga library, they can easily be generated within `ga_prdxpr/src_prdxpr` as needed and added with minimal effort.

As mentioned before, we distinguish inner, interior, outer and geometric products.

- Inner ($a \cdot b$) and interior products, like contractions and expansions ($a \vee b^\star$, $a \wedge b^\star$), are grade reducing operations. The inner product combines arguments of the same type and grade, while the interior products involve duals of one of their arguments, combining and linking arguments of different grades. Inner products are linked to the metric of the modelled space.

- The outer product ($a \wedge b$) is a grade-increasing operation adding the grades of its arguments. Outer products, like the wedge product, connect subspaces represented by their operands and create new subspaces defined by the span of the subspaces of the operands. They establish a join-operation between the involved subspaces.

- The geometric product ($a \wedge\!\!\!\wedge b$) is the combination of both types of products in one expression. They can result in multivectors consisting of parts with different grades as a result of the operation. The main advantage of the geometric product is invertibility under certain circumstances.

- Regressive products ($a \circ b \equiv \underline{\bar{a} \cdot \bar{b}}$, $a \vee b \equiv \underline{\bar{a} \wedge \bar{b}}$, $a \vee\!\!\!\vee b \equiv \underline{\bar{a} \wedge\!\!\!\wedge \bar{b}}$) use complement operations on their arguments before and after applying the operation[1]. The regressive wedge product creates an intersection of the spaces used by their arguments, thus effectively creating a meet-operation of between the involved spaces.

- Sandwich products ($R \wedge\!\!\!\wedge u \wedge\!\!\!\wedge \tilde{R}$ for EGA, $M \vee\!\!\!\vee u \vee\!\!\!\vee \tilde{M}$ for PGA) are used to implement orthogonal transformations of geometric objects. They realize rotations for EGA and general motions including rotation and translation for PGA by concatenating two consecutive reflections, used as primitives to realize the resulting transformations. This can be achieved algebraically by multiplying the object symmetrically from the left- and right-hand side with the appropriate objects, we either call rotors R in EGA or motors M in PGA.

[1] for definition of the left and right complement operations marked with underbar and overbar, see next section

Following bi-argument operations $op(a,b) = a\ op\ b$ and their regressive variants $rop(a,b) = \underline{\bar{a}\ op\ \bar{b}}$ are implemented (with `op(a,b)` or `rop(a,b)` as the name of the implemented function). Using A vs. a implies $gr(A) \geq gr(a)$, where $gr(arg)$ returns the grade of the argument arg:

- Inner product, delivering a scalar: $a \cdot b$, `dot(a,b)`
- Outer product of a j- and a k-vector, delivering a $j+k$-vector in $span(a,b)$: $a \wedge b$, `wdg(a,b)`
- Geometric product: $a \wedge\!\!\!\wedge b$, `operator*(a,b)`
  - For vectors a and b: $a \wedge\!\!\!\wedge b = a \cdot b + a \wedge b$
  - For a vector a and a multivector B: $a \wedge\!\!\!\wedge B = B \gg a + a \wedge B$
  - For a multivector A and a vector b: $A \wedge\!\!\!\wedge b = b \ll A + A \wedge b$
- Commutator product (the asymmetric part of the geometric product): $cmt(a,b) \equiv \frac{1}{2}(a \wedge\!\!\!\wedge b - b \wedge\!\!\!\wedge a)$,
- Regressive commutator product (the asymmetric part of the regressive geometric product):
  $rcmt(a,b) \equiv \frac{1}{2}(\underline{\bar{a} \wedge\!\!\!\wedge \bar{b} - \bar{b} \wedge\!\!\!\wedge \bar{a}}) = \frac{1}{2}(a \vee\!\!\!\vee b - b \vee\!\!\!\vee a)$, `rcmt(a,b)`
- Regressive inner product, delivering a pseudoscalar: $a \circ b$, `rdot(a,b)`
- Regressive outer product: $a \vee b$, `rwdg(a,b)`
- Regressive geometric product: $a \vee\!\!\!\vee b$, `rgpr(a,b)`
- Left (bulk) contraction: $a \rfloor B = a \ll B = a_\star \vee B$, `operator<<(a,B)`, which implements `left_bulk_contract(a,B)`
- Right (bulk) contraction: $B \lfloor a = B \gg a = B \vee a^\star$, `operator>>(B,a)`, which implements `right_bulk_contract(B,a)`
- Left weight contraction: $a_\star \vee B$, `left_weight_contract(a,B)`
- Right weight contraction: $B \vee a^\star$, `right_weight_contract(B,a)`
- Left bulk expansion: $A_\star \wedge b$, `left_bulk_expand(A,b)`
- Right bulk expansion: $a \wedge B^\star$, `right_bulk_expand(a,B)`
- Left weight expansion: $A_\star \wedge b$, `left_weight_expand(A,b)`
- Right bulk expansion: $a \wedge B^\star$, `right_weight_expand(a,B)`

where the items from the left contraction downwards are all considered interior products. They operate directly on the subspaces occupied by the objects. The product operation is performed after applying a dualization operation to one operand.

The product tables for all products are generated in `ga_prdxpr`, and coefficient expressions for specific argument types like scalars, vector, bivectors, etc. can be generated based on those product tables using configuration options of `ga_prdxpr`.

1.2 Basics

The following is provided for Euclidean geometric algebra of two-dimensional space (*ega2d*) using an orthormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \text{ and } \boldsymbol{e}_2^2 = +1$$

$$s_{2d} = s\,\mathbf{1} \tag{1a}$$

$$v_{2d} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 \tag{1b}$$

$$ps_{2d} = ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1} \tag{1c}$$

equation (1a) is the scalar part s_{2d}, equation (1b) contains the vector part v_{2d}, and equation (1c) contains the pseudoscalar part ps_{2d}. The index $2d$ is typically omitted when clear from context. The four basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_{12}\}$. Using this basis a multivector M of $2d$-space is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + ps\,\boldsymbol{e}_{12} \tag{2}$$

where equation (2) contains three parts: the scalar part $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$; if $\mathbf{1}$ is not shown, it is implicitly assumed for scalar values), the vector part v (basis elements $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$) and the pseudoscalar part $ps\,\boldsymbol{e}_{12} = ps\,\mathbb{1}$ (basis element is $I_{2d} = \boldsymbol{e}_{12}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{2d} is a bivector in two-dimensional Euclidean space).

For Euclidean geometric algebra of three-dimensional space (*ega3d*) using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = +1$$

$$s_{3d} = s\,\mathbf{1} \tag{3a}$$

$$v_{3d} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 \tag{3b}$$

$$B_{3d} = b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \tag{3c}$$

$$ps_{3d} = ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1} \tag{3d}$$

equation (3a) is the scalar part s_{3d}, equation (3b) is the vector part v_{3d}, equation (3c) is the bivector part B_{3d}, and equation (3d) is the pseudoscalar part ps_{3d}. The index $3d$ is typically omitted when clear from context. Compared to the $2d$-case it is obvious, that all parts depend on context, specifically on the dimensionality of the modelled space, and thus need to be defined and treated accordingly.[2] The basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{123}\}$.

[2] *hint*: Since C++ is a statically typed language, those types need to be well-defined and distinguishable from each other. This is especially important for the different scalar and pseudoscalar types. The ga library uses type tagging extensively for that purpose.

Using this basis a multivector M of $3d$-space is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{123} \tag{4}$$

where equation (4) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the bivector part B (basis elements $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{123} = ps\,\mathbb{1}$ (basis element is $I_{3d} = \boldsymbol{e}_{123}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. I_{3d} is a trivector in three-dimensional Euclidean space).

Projective geometric algebra of two-dimensional Euclidean space (*pga2dp*) is modelled by embedding it in a three-dimensional space. Using an orthonormal basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \text{ and } \boldsymbol{e}_3^2 = 0$$

$$s_{2dp} = s\,\mathbf{1} \tag{5a}$$

$$v_{2dp} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 \tag{5b}$$

$$B_{2dp} = b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \tag{5c}$$

$$ps_{2dp} = ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1} \tag{5d}$$

The most obvious difference is the basis vector $\boldsymbol{e}_3$ that squares to zero and the pseudoscalar of that space that is chosen to be $\boldsymbol{e}_{321}$. The trivector $\boldsymbol{e}_{321}$ squares to 0, because of it's component $\boldsymbol{e}_3$ contained as a factor. Equation (5a) is the scalar part s_{2dp}, equation (5b) is the vector part v_{2dp}, equation (5c) is the bivector part B_{2dp}, and equation (5d) is the pseudoscalar part ps_{2dp}. The index $2dp$ is typically omitted when clear from context. The eight basis elements are $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{321}\}$. Using this basis a multivector M of two-dimensional projective space ($2dp$) is defined as

$$M = s\,\mathbf{1} + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + b_1\boldsymbol{e}_{23} + b_2\boldsymbol{e}_{31} + b_3\boldsymbol{e}_{12} + ps\,\boldsymbol{e}_{321} \tag{6}$$

where equation (6) contains four parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$ and $\boldsymbol{e}_3$), the bivector part B (basis elements $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$ and $\boldsymbol{e}_{12}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{321} = ps\,\mathbb{1}$ (basis element is $I_{2dp} = \boldsymbol{e}_{321}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a trivector in two-dimesional Euclidean projective space that squares to zero).

Projective geometric algebra of three-dimensional Euclidean space (*pga3dp*) is modelled by embedding it in a four-dimensional space. Using an orthonormal

basis $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$ there are following basis vectors:

$$(\boldsymbol{e}_1)^2 = \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 = \boldsymbol{e}_1 \cdot \boldsymbol{e}_1 = \boldsymbol{e}_1^2 = +1, \boldsymbol{e}_2^2 = +1, \boldsymbol{e}_3^2 = +1, \text{ and } \boldsymbol{e}_4^2 = 0$$

$$s_{3dp} = s\,\mathbf{1} \tag{7a}$$

$$v_{3dp} = v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + v_4\boldsymbol{e}_4 \tag{7b}$$

$$B_{3dp} = b_{41}\boldsymbol{e}_{41} + b_{42}\boldsymbol{e}_{42} + b_{43}\boldsymbol{e}_{43} + b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \tag{7c}$$

$$t_{3dp} = t_{423}\boldsymbol{e}_{423} + t_{431}\boldsymbol{e}_{431} + t_{412}\boldsymbol{e}_{412} + t_{321}\boldsymbol{e}_{321} \tag{7d}$$

$$ps_{3dp} = ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1} \tag{7e}$$

Now we have a basis vector $\boldsymbol{e}_4$ that squares to zero. The bivector has six components, and there is a trivector t. The pseudoscalar $\boldsymbol{e}_{1234}$ is a quadvector that squares to 0, because of it's component $\boldsymbol{e}_4$ contained as a factor. Equation (7a) is the scalar part s_{3dp}, equation (7b) is the vector part v_{3dp}, equation (7c) is the bivector part B_{3dp}, equation (7d) is the trivector part t_{3dp}, and equation (7e) is the pseudoscalar part ps_{3dp}. The index $3dp$ is typically omitted when clear from context. The sixteen basis elements of $pga3d$ are: $\{\mathbf{1}, \boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3, \boldsymbol{e}_4, \boldsymbol{e}_{41}, \boldsymbol{e}_{42}, \boldsymbol{e}_{43}, \boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}, \boldsymbol{e}_{423}, \boldsymbol{e}_{431}, \boldsymbol{e}_{412}, \boldsymbol{e}_{321}, \boldsymbol{e}_{1234}\}$. A multivector M of three-dimensional projective space ($3dp$) is defined as

$$\begin{aligned} M = {} & s\,\mathbf{1} \\ & + v_1\boldsymbol{e}_1 + v_2\boldsymbol{e}_2 + v_3\boldsymbol{e}_3 + v_4\boldsymbol{e}_4 \\ & + b_{41}\boldsymbol{e}_{41} + b_{42}\boldsymbol{e}_{42} + b_{43}\boldsymbol{e}_{43} + b_{23}\boldsymbol{e}_{23} + b_{31}\boldsymbol{e}_{31} + b_{12}\boldsymbol{e}_{12} \\ & + t_{423}\boldsymbol{e}_{423} + t_{431}\boldsymbol{e}_{431} + t_{412}\boldsymbol{e}_{412} + t_{321}\boldsymbol{e}_{321} \\ & + ps\,\boldsymbol{e}_{1234} \end{aligned} \tag{8}$$

where equation (8) contains five parts: the scalar component $s\,\mathbf{1}$ (basis element is the scalar $\mathbf{1}$), the vector part v (basis elements $\boldsymbol{e}_1$, $\boldsymbol{e}_2$, $\boldsymbol{e}_3$, $\boldsymbol{e}_4$), the bivector part B (basis elements $\boldsymbol{e}_{41}$, $\boldsymbol{e}_{42}$, $\boldsymbol{e}_{43}$, $\boldsymbol{e}_{23}$, $\boldsymbol{e}_{31}$, $\boldsymbol{e}_{12}$), the trivector part t (basis elements $\boldsymbol{e}_{423}$, $\boldsymbol{e}_{431}$, $\boldsymbol{e}_{412}$, $\boldsymbol{e}_{321}$) and the pseudoscalar part $ps\,\boldsymbol{e}_{1234} = ps\,\mathbb{1}$ (basis element is $I_{3dp} = \boldsymbol{e}_{1234}$, which is sometimes written as $\mathbb{1}$ to show its character as pseudoscalar of this space. It is a quadvector in three-dimensional Euclidean projective space that squares to zero).

It is important to note that it is seldomly required to use a full multivector in the respective algebra. For most applications it is sufficient to use a blade or k-vector exclusively, which represents one grade of the multivector, e.g. a vector or a bivector. This reduces the effort in terms of memory and computational requirements significantly – typically reduced to a level that is required to to solve the underlying application problem anyway.

The complement operation used in the library is defined in [Lengyel, 2024b]. It is uniquely determined with respect to the outer product (not with respect to the geometric product, as commonly used for dualization by many autors working with GA). This is essential to generate consistent signs of expressions, operators, complements and duals. The left complement $\underline{u}$ and the right complement $\bar{u}$ are defined as

$$\underline{u} \wedge u = I_n \tag{9a}$$

$$u \wedge \bar{u} = I_n \tag{9b}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n$ as the pseudoscalar of the n-dimensional algebra, and the wedge symbol ($\wedge$) as the operator symbol of the outer product. The effect of the complement is to turn basis elements not contained in the object u into the constituting basis elements of the complement of the object $\underline{u}$ or $\bar{u}$. Connecting the object and its complement with the wedge products creates an object that fills the available space in the sense of requiring all basis elements of the space to represent it. The full space itself is represented by the pseudoscalar of that space, since the pseudoscalar I_n is formed formed by the product of all n basis vectors of the space. An implication of the definition is that the complement ($\underline{u}$ or $\bar{u}$) is orthogonal to u.

It can be shown that the left and right complements fulfill

$$\underline{\bar{a}} = a \tag{10}$$

thus the left and right complement operations are inverses to each other. This is valid independent of the sequence of application, independent of the grade of the argument a, and independent of the dimension of the space modelled in the algebra. Based on this it is possible to define so-called regressive products using rules analog to deMorgan's rules in boolean algebra as follows (definitions can be found in [Lengyel, 2024b] and in [Browne, 2012]):

$$a \vee b \equiv \underline{\bar{a} \wedge \bar{b}} \tag{11a}$$

$$a \vee b \equiv \overline{\underline{a} \wedge \underline{b}} \tag{11b}$$

The same complement is applied to both arguments first, the resulting expressions are combined with the wedge product, and the inverse complement operation is used to transform the outcome into the final result, the regressive product. The inverted wedge symbol ($\vee$) is used for the regressive wedge product. Both definitions (11a) and (11b) are equivalent. Equation (11a) is used to derive expressions for regressive products in `ga_prdxpr`.

There are some additional formulas for complements which are useful for working with GA expressions and thus are provided here for reference. Taking the right and left complements of (11a) and (11b) we end up with

$$\overline{a \vee b} = \overline{\underline{\bar{a} \wedge \bar{b}}} = \bar{a} \wedge \bar{b} \tag{12a}$$

$$\underline{a \vee b} = \overline{\underline{\underline{a} \wedge \underline{b}}} = \underline{a} \wedge \underline{b} \tag{12b}$$

Substituting $a = \underline{u}$ and $b = \underline{v}$ into (12a) and $a = \bar{u}$ and $b = \bar{v}$ into (12b) and exchanging the left and right hand sides of the equations we end up with

$$u \wedge v = \overline{\underline{u} \vee \underline{v}} \tag{13a}$$

$$u \wedge v = \underline{\bar{u} \vee \bar{v}} \tag{13b}$$

which shows that the outer and the regressive outer product can be expressed by each other.

The complement operation is used to define a unique dualization operation that works for cases with non-degenerate as well as for cases with degenerate metric, like in projective geometric algebra PGA. This is the main advantage compared to the usual prodedure of dualization by multiplying the object with the pseudoscalar (or its inverse) using the geometric product, since the latter breaks down for spaces with degenerate metrics.

The metric is defined by the dot product of the basis vectors as

$$\mathfrak{g}_{ij} \equiv \boldsymbol{e}_i \cdot \boldsymbol{e}_j \tag{14}$$

It is a square matrix that is the identity matrix I_n for EGA, since all basis vectors square to $\boldsymbol{e}_i \cdot \boldsymbol{e}_i = +1$, and $\boldsymbol{e}_i \cdot \boldsymbol{e}_j = 0$ for $i \neq j$. It is invertible in this case. However, for PGA one basis vector squares to zero, and thus there is one zero on the main diagonal of the corresponding matrix of the metric, which is therefore not invertible. It is called degenerate for that reason.

The extended metric $\mathbf{G}$ is a so-called exomorphism matrix that fulfills

$$\mathbf{G}(A \wedge B) = \mathbf{G}(A) \wedge \mathbf{G}(B) \tag{15}$$

for any multivector A and B (see [Lengyel, 2024b], pp. 61ff.).
Regarding $\mathbf{G}(u) = \mathbf{G}u$ as a unary operation on u we can define the corresponding regressive operation by first taking the complement of the argument and then applying the opposed complement on the result. This defines the regressive metric $\mathbb{G}$ (named anti-metric by Lengyel). It is defined as

$$\mathbb{G}u \equiv \underline{\mathbf{G}\overline{u}} = \overline{\mathbf{G}\underline{u}} \tag{16}$$

$\mathbf{G}$ and $\mathbb{G}$ are related by

$$\mathbf{G}\mathbb{G} = det(\mathfrak{g})I_n \tag{17}$$

Thus, if all basis vectors square to ± 1, we have $\mathbf{G}\mathbb{G} = \pm I_n$, and both are invertible. However, if one basis vector squares to zero, we get $\mathbf{G}\mathbb{G} = 0$, which means that none of them is invertible, i.e. we have a degenerate metric.

For $\mathbb{G}$ there is the relation

$$\mathbb{G}(A \vee B) = \mathbb{G}(A) \vee \mathbb{G}(B) \tag{18}$$

that is complementary to equation (15).

In PGA the product $\mathbf{G}u$ defines the bulk of of an object u. It is defined as

$$u_\bullet = \mathbf{G}u \tag{19}$$

The bulk only contains components that *do not* contain the projective dimension ($\boldsymbol{e}_3$ for $pga2dp$ or $\boldsymbol{e}_4$ for $pga3dp$) as a factor. It contains positional information on u relative to the origin. Zero bulk means that u contains the origin.

In PGA the product $\mathbb{G}u$ defines the weight of of an object u. It is defined as

$$u_\circ = \mathbb{G}u \tag{20}$$

The weight selects components that *do* contain the projective dimension ($\boldsymbol{e}_3$ for $pga2dp$ or $\boldsymbol{e}_4$ for $pga3dp$) as a factor. It contains directional information on u, i.e. its attitude and direction independent of its position. Zero weight means that u is contained in the horizon, i.e. it is infinitely far away.

This effective split for each object into bulk and weight in projective algebras can be written as

$$u = u_\bullet + u_\circ \tag{21}$$

This split is also relevant for the corresponding norms

$$\|u\| = \|u\|_\bullet + \|u\|_\circ = \sqrt{A \cdot A} + \sqrt{A \circ A} \tag{22}$$

where the left hand side is called the geometric norm and the right hand side consists of the sums of the bulk and weight norms. The bulk norm returns a scalar value of the corresponding algebra, whereas the weight norm provides a pseudoscalar value of the respective algebra. Consequently, the geometric norm is an object consisting of two distinct parts, defined here as

a dual number in the corresponding algebra. When we apply unitization to geometric objects we devide the object in such a way that the magnitude of the weight norm becomes a value of one. Both, bulk and weight norms, have a positive argument under the square root due to the definition of the inner product and the regressive inner product.

Using the metric and the right complement the right bulk dual $A^\star$ is defined as

$$A^\star = \overline{\mathbf{G}A} = \overline{A_\bullet} \tag{23}$$

where A is an arbitrary multivector, $\mathbf{G}$ the extended metric and $\mathbf{G}A$ a matrix-vector-product between them. $A^\star$ is also called the Hodge dual. The multiplication with the metric happens before the application of the complement operation.

There is also a corresponding left dual $A_\star$ operation using the metric and the left complement. It is defined as

$$A_\star = \underline{\mathbf{G}A} = \underline{A_\bullet} \tag{24}$$

The Hodge star (symbolized by a filled star $\star$) is used as dualization operator in both cases. The lower or upper position symbolizes the left and right application. The difference between the hodge dual and the complement is in both cases that multiplying the multivector with the extended metric happens *before* taking the corresponding complement. However, this does not lead to different results in spaces where the metric is the identity matrix. It only yields different results in spaces with a more involved metric, e.g. like in projective spaces with a non-invertible (degenerate) metric as we have in PGA.

The effect of applying the metric before taking the complement is equivalent to applying the complement operation on the bulk part of the object u exclusively, and not on the full object u. Of course, the dualization can be applied to the weight part as well. This operation (symbolized by an unfilled star "$\star$") uses the regressive extended metric and is defined for the right weight dual $A^\star$ using the right complement as

$$A^\star = \overline{\mathbb{G}A} = \overline{A_\circ} \tag{25}$$

for an arbitrary multivector A. The corresponding left weight dual $A_\star$ is defined as

$$A_\star = \underline{\mathbb{G}A} = \underline{A_\circ} \tag{26}$$

using the left complement on the weight part of A.

Relations to the geometric product and the regressive geometric product are given by

$$A^{\star} = \tilde{A} \wedge I_n = \tilde{A} \wedge \mathbb{1} \tag{27a}$$
$$A_{\star} = I_n \wedge \tilde{A} = \mathbb{1} \wedge \tilde{A} \tag{27b}$$
$$A^{\star\star} = \underset{\sim}{A} \vee \mathbf{1} \tag{27c}$$
$$A_{\star\star} = \mathbf{1} \vee \underset{\sim}{A} \tag{27d}$$

with $I_n = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2 \wedge \ldots \wedge \boldsymbol{e}_n = \mathbb{1}$ as the pseudoscalar of the n-dimensional space. $\tilde{A}$ is the reversion operation, and $\underset{\sim}{A}$ is the regressive reversion operation applied to A (e.g. with $A = \boldsymbol{e}_1 \wedge \boldsymbol{e}_2$ the reverse is $\tilde{A} = \boldsymbol{e}_2 \wedge \boldsymbol{e}_1$), and "$\wedge$" and "$\vee$" are the symbols for the geometric and regressive geometric products.

Following identities link the outer product and the inner product. They hold for operands of same grade:

$$A \wedge B^{\star} = (A \cdot B) I_n, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{28a}$$
$$A_{\star} \wedge B = (A \cdot B) I_n, \quad \text{when } \texttt{gr(A)} = \texttt{gr(B)} \tag{28b}$$

The influence of the metric for the dualization operation can be seen in its effect for the different algebras which also depends on the dimension of the modelled space:

- *ega2d*: $A_{\star}$ = `left_dual(A)`, $A^{\star}$ = `right_dual(A)`
- *ega3d*: $A_{\star}$ = `dual(A)`, $A^{\star}$ = `dual(A)`
- *pga2dp*: $A_{\star}$ = `bulk_dual(A)`, $A^{\star}$ = `bulk_dual(A)`
- *pga2dp*: $A_{\star\star}$ = `weight_dual(A)`, $A^{\star\star}$ = `weight_dual(A)`
- *pga3dp*: $A_{\star}$ = `left_bulk_dual(A)`, $A^{\star}$ = `right_bulk_dual(A)`
- *pga3dp*: $A_{\star\star}$ = `left_weight_dual(A)`, $A^{\star\star}$ = `right_weight_dual(A)`

In even-dimensional spaces (*ega2d*, *pga3dp*) we have to distinguish left and right operations, while in odd-dimensional spaces (*ega3d*, *pga2dp*) we don't, since they produce the same result. The "left" operations are typically used on the left-hand side of the binary operation and the "right" operations on the right-hand side of the product operator. The only deviation from that pattern occurs for the definition of the regressive operations, as defined in (11a)

or (11b).

In a two-dimensional algebra, like *ega2d*, the effect of the right dual of a vector u is to turn the vector by +90°, i.e. into the orientation of $\boldsymbol{e}_{12}$, while the left dual turns the vector by −90°, i.e. into the direction against the orientation of $\boldsymbol{e}_{12}$. This statement is valid for both right- and left-handed coordinate systems.

The multiplicative identity of the geometric product is the scalar $\mathbf{1}$:

$$\mathbf{1} \wedge a = a \wedge \mathbf{1} = a \tag{29}$$

The geometric product combines the dimensions that are present in an object representation, e.g. for *pga2dp*:

$$\begin{aligned} \boldsymbol{e}_1 \wedge \boldsymbol{e}_1 &= 1 \\ \boldsymbol{e}_2 \wedge \boldsymbol{e}_2 &= 1 \\ \boldsymbol{e}_3 \wedge \boldsymbol{e}_3 &= 0 \end{aligned} \tag{30}$$

The multiplicative identity of the regressive geometric product is the pseudoscalar $\mathbb{1} = I_n$ of the respective algebra:

$$\mathbb{1} \vee a = a \vee \mathbb{1} = a \tag{31}$$

The regressive geometric product combines the dimensions that are <u>not</u> present in an object representation, e.g. for *pga2dp*:

$$\begin{aligned} \overline{\boldsymbol{e}_1} \vee \overline{\boldsymbol{e}_1} &= \mathbb{1} \\ \overline{\boldsymbol{e}_2} \vee \overline{\boldsymbol{e}_2} &= \mathbb{1} \\ \overline{\boldsymbol{e}_3} \vee \overline{\boldsymbol{e}_3} &= 0 \end{aligned} \tag{32}$$

where eqs. (32) can be shown to be equivalent to eqs. (30) by taking the left complement of both sides and applying the complement definition (11a) to it.

The inner product between multivectors A and B is defined as

$$A \cdot B = A^T \mathbf{G} B \tag{33}$$

with $\mathbf{G}$ as the extended metric and matrix multiplication on the right hand side. It satisfies the identity

$$A \cdot B = \langle A \wedge \tilde{B} \rangle_0 = \langle B \wedge \tilde{A} \rangle_0 \tag{34}$$

where $\wedge\!\!\!\wedge$ stands for the geometric product within the grade projection operator $\langle\rangle_k$ for grade k.

The contraction is explicitly defined as

$$A\rfloor B = A \ll B = A_\star \vee B \tag{35a}$$
$$B\lfloor A = B \gg A = B \vee A^\star \tag{35b}$$

with $\star$ denoting the Hodge dual which is formed by the respective complement operation (in spaces of even dimension the left or right complement respectively, and in spaces of odd dimension the complement function regardless of the side of the operand). For blades A and B, they satisfy

$$A\rfloor B = A \ll B = \langle B \wedge\!\!\!\wedge \tilde{A}\rangle_{gr(B)-gr(A)} \tag{36a}$$
$$B\lfloor A = B \gg A = \langle \tilde{A} \wedge\!\!\!\wedge B\rangle_{gr(B)-gr(A)} \tag{36b}$$

and fulfill the requirement that $A\rfloor B = A\lfloor B = A \cdot B$ whenever A and B have the same grade. In this case the contractions reduce to the inner product.

The geometric product between and vector v and a blade B is defined in terms of interior and exterior products, i.e. the right contraction $\lfloor$, the left contraction $\rfloor$ and wedge product $\wedge$ as follows:

$$v \wedge\!\!\!\wedge B = B\lfloor v + v \wedge B \tag{37a}$$
$$B \wedge\!\!\!\wedge v = v\rfloor B + B \wedge v \tag{37b}$$

or sometimes expressed using the right shift ($\gg$) or left shift ($\ll$) operators for the contractions (as implemented in the ga library)

$$v \wedge\!\!\!\wedge B = B \gg v + v \wedge B \tag{38a}$$
$$B \wedge\!\!\!\wedge v = v \ll B + B \wedge v \tag{38b}$$

or sometimes by exclusively using the operations of the exterior algebra (wedge product "$\wedge$", bulk dual "$\star$" and regressive wedge product "$\vee$")

$$v \wedge\!\!\!\wedge B = B \vee v^\star + v \wedge B \tag{39a}$$
$$B \wedge\!\!\!\wedge v = v_\star \vee B + B \wedge v \tag{39b}$$

as an alternative. Each one is equivalent.

Every vector v can be decomposed into parts $v_\parallel$ parallel to the k-blade B and $v_\perp$ perpendicular to B, such that $v = v_\parallel + v_\perp$. For equation (38a): $B \gg v$

is a $(k-1)$-blade. If $v_{\parallel} \neq 0$ then $B \gg v$ represents a subspace of B. $v \wedge B$ is a $(k+1)$-blade. If $v_{\perp} \neq 0$ then $v \wedge B$ represents $span(v, B)$.
For arguments of equal grade the contractions reduce to the dot product, so that one can write specifically for two vectors a and b:

$$a \wedge\!\!\!\wedge b = a \cdot b + a \wedge b \tag{40}$$

[Lengyel, 2025] has shown that the geometric product can be derived by using operations from Grassmann algebra exclusively. These are the wedge product, the complement and the duality operations. This demonstrates that the geometric product is not a fundamental operation in its own right, but can be derived from more fundamental basic operations.

For that purpose Lengyel introduced the transwedge product of order k by following definition:

$$a \underset{k}{\wedge\!\!\!\wedge} b = \sum_{c \in \mathcal{B}_k} (\underline{c} \vee a) \wedge (b \vee c^{\star}) \tag{41}$$

where $\mathcal{B}_k$ is the set of all basis elements of order k. For *ega3d* it means for example $\mathcal{B}_0 = \{1\}$, $\mathcal{B}_1 = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3\}$, $\mathcal{B}_2 = \{\boldsymbol{e}_{23}, \boldsymbol{e}_{31}, \boldsymbol{e}_{12}\}$, and $\mathcal{B}_3 = \{\boldsymbol{e}_{123}\}$. From these expressions the geometric product can be formed by

$$a \wedge\!\!\!\wedge b = \sum_{k=0}^{n} (-1)^{k(k-1)/2} a \underset{k}{\wedge\!\!\!\wedge} b \tag{42}$$

For $k = 0$ the formula yields the outer product (wedge product). For higher values of k other products are generated which sum up to the full geometric product. Thus the transwedge products helps to decompose the geometric product. The most interesting for applications are the wedge product resulting for $k = 0$ and the transwedge product of order $k = 1$.

The regressive wedge product in equation (41) can be replaced by the wedge product using the complement-based definitions given in equations (10) to (13b). That means for example that equation (41) can be transformed to the equivalent expression

$$a \underset{k}{\wedge\!\!\!\wedge} b = \sum_{c \in \mathcal{B}_k} (\underline{c \wedge \bar{a}}) \wedge (\underline{\bar{b} \wedge \overline{c^{\star}}}) \tag{43}$$

expressing it by using the wedge product exclusively.

Since every product can by assigned a corresponding regressive product in analogy to definition (11a) or (11b), the transwedge product (41) also has a regressive form:

$$a \underset{k}{\vee\!\!\!\vee} b \equiv \overline{\underline{a} \underset{k}{\wedge\!\!\!\wedge} \underline{b}} = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c} \vee \underline{a}) \wedge (\underline{b} \vee c^{\star})} \tag{44}$$

which in turn can be expressed in terms of the wedge product exclusively as:

$$a \underset{k}{\vee\!\!\!\wedge} b = \sum_{c \in \mathcal{B}_k} \overline{(\underline{c \vee a}) \wedge (\underline{b \vee \overline{c^\star}})} \tag{45}$$

Equations (43) and (45) are used for calculation of the product tables of the (regressive) transwedge products of order k and the resulting (regressive) geometric products in `ga_prdxpr`. The transwedge product is very useful to show that the geometric product can be derived from primitives of Grassmann algebra. When (regressive) transwedge product expressions are calculated, it can be shown that each transwedge product and the resulting coefficient expressions are equivalent to specific combinations of wedge-, commutator-, dot-products, and their regressive counterparts.

1.3 Physics modelling

The key idea is to represent physical entities using objects that are represented directly in the underlying algebra. They have a geometric interpretation and can be directly manipulated by applying algebraic operations.

EGA is limited to objects that contain the origin. It can be used to directly model vectors (i.e. directions), lines, and planes through the orgin. In addition, it can model complex numbers, quaternions and thus rotations in $2d$ and $3d$ well. To realize them, it uses sandwich products for calculating the transformation resulting from two consequtive reflections either across lines ($2d$-case) or across planes ($3d$-case). The resulting transformations are rotations. The advantage of using geometric algebra to model those rotations is the intuitive modelling, due to geometric meaning of the objects and the fact that the sandwich products can be used to transform different objects using the same formulas regardless of the type of object to be transformed.

In additon, *PGA* is capable to model objects that do not contain the origin. Thus it can be used to directly model points, directions, lines, and planes that may or may not contain the origin. The additional degree of freedom is possible due to embedding the Euclidean space of dimension n into a projective space of dimension $n + 1$, e.g. $2d$-Euclidean algebra is modelled by embedding it in a $3d$-projective space, and $3d$-Euclidean algebra is modelled in a $4d$-projective space. In a projective space the objects must be projected into the modelled space. Thus the objects of the embedding space typically have one dimension more than the corresponding objects in modelled space. E.g. a projective point (a scalar or $0d$-object) is a vector in the embedding space (a $1d$-object), and a bivector in projective space (a $2d$-object) is used to model a line (a $1d$-object). The projection reduces the object dimension by one. Projective objects can be multiplied by a scalar without changing the geometrical meaninig of the projective object in the embedding space.

Due to its extended expressiveness *PGA* is suitable to model physics of mass points and rigid bodies. It can model position, velocity, acceleration, force, torque, momentum, angular momentum, etc. to describe kinematics and dynamics of mass points and rigid bodies. The approach used here is inspired by [Dorst and De Keninck, 2022] and [Dorst and De Keninck, 2023], but it does not use plane-based *PGA* with planes as vectors, which are based on dual representations of the objects to be modelled. It rather uses the approach as described by [Lengyel, 2024b] using projective geometric algebra based on direct representations of points, lines and planes. The reasons for

choosing the latter approach are explained in [Lengyel, 2024a]. Further hints on inconsistencies in typical GA application up-to-now can be found as well in [Kritchevsky, 2024]. The goal of the this work is to be as consistent in terms of mathematical application of GA and as intuitively applicable as possible.

1.4 Literature

References

[Browne, 2012] Browne, J. (2012). *Grassmann Algebra*, volume 1: Foundations. Barnard Publishing.

[Dorst and De Keninck, 2023] Dorst, L. and De Keninck, S. (August 15, 2023). May the forque be with you - dynamics in pga. *University of Amsterdam, The Netherlands*, V2.6.

[Dorst and De Keninck, 2022] Dorst, L. and De Keninck, S. (March 14, 2022). A guided tour to the plane-based geometric algebra pga. *University of Amsterdam, The Netherlands*, V2.0.

[Kritchevsky, 2024] Kritchevsky, A. (February 28, 2024). The case against geometric algebra. *Alex Kritchevsky's blog.*

[Lengyel, 2024b] Lengyel, E. (2024b). *Projective Geometric Algebra Illuminated.* Terathon Software LLC.

[Lengyel, 2024a] Lengyel, E. (August 23, 2024a). Poor Foundations in Geometric Algebra. *Eric Lengyel's blog.*

[Lengyel, 2025] Lengyel, E. (May 23, 2025). The Transwedge Product. *Eric Lengyel's blog.*